

Final Report

BARBIER Nathan, ERASMUS STUDENT

I. INTRODUCTION

The increasing complexity of mobile networks, especially with the rise of 5G and LTE technologies, has led to a growing need for spatial analysis of network performance. Operators and regulators must understand how key metrics such as signal strength, data rate, and latency vary across geographic regions to optimize coverage and ensure quality of service.

This project explores the use of deep learning for predicting spatial patterns in mobile network performance. Using real-world data collected from the RTR Open Data Portal in Austria, we aim to model how different performance indicators interact across space and time. The dataset includes measurements from multiple mobile operators (A1, Magenta, Drei) and covers various technologies and levels of location accuracy.

The core objective is to develop a learning-based approach that can estimate one or more performance metrics across a spatial grid, based on existing measurements. This work focuses on the potential of convolutional architectures in particular, the U-Net to learn from sparse measurement data and generate high-resolution predictions over urban areas.

II. METHODS

A. Data Acquisition from RTR API

The raw dataset was collected through the RTR (Rundfunk und Telekom Regulierungs-GmbH) Open Data Portal, which offers publicly accessible and anonymized measurements of mobile network performance across Austria. The measurements are crowd-sourced from real users via the RTR NetTest mobile application.

To ensure geographic consistency, we restricted our study to measurements taken within the city of Vienna, a dense urban area with high network activity. To further guarantee data quality, we applied strict filtering conditions during data acquisition.

A custom script was developed to query the RTR API using the following criteria:

Device platform: Only measurements collected from devices labeled with iOS, Android, or iPhone were kept, ensuring data originated from standard consumer smartphones.

Network provider: We focused exclusively on data from the provider labeled "A1 TA" (A1 Telekom Austria), to maintain consistency across network conditions.

Availability: Samples with incomplete or missing values for essential metrics were excluded.

Technology & accuracy: The API was queried for high-accuracy measurements using LTE or 5G technologies only. This filtering process helped eliminate noisy or inconsistent samples and ensured that the dataset was suitable for reliable spatial modeling.

The Python filtering logic used to enforce these conditions is shown in Figure 1.

```
df = df[available]

device_cond = (
    df['platform'].str.contains("iOS", case=False, na=False) |
    df['platform'].str.contains("Android", case=False, na=False) |
    df['model'].str.contains("iPhone", case=False, na=False)
)

provider_cond = df['provider_name'].str.contains("A1 TA", case=False, na=False)
```

Figure 1 – Code snippet showing filtering conditions for platform type (iOS/Android), model (iPhone), and provider (A1 TA). These constraints help retain only high-quality and consistent measurement samples

B. Dataset and Preprocessing

After filtering, the raw geolocated measurement points are projected onto a fixed-size 2D spatial grid. Each cell in the grid represents a small square region of real-world space, and aggregates all available measurements within its bounds.

To prepare the data for deep learning, we perform the following preprocessing steps:

1. Spatial Gridding and Interpolation

Each selected area (e.g., 150×150 or 300×300 pixels) is divided into a regular grid. For each cell, the value of each metric (download, upload, ping, signal strength) is computed using interpolation from nearby measurements. We apply linear interpolation for smooth metrics (e.g., throughput), and nearest-neighbor interpolation for sparse ones.

2. Patch Extraction

Once the full grid is constructed, we extract overlapping square patches from it. Each patch has:

A fixed size of 16×16 pixels

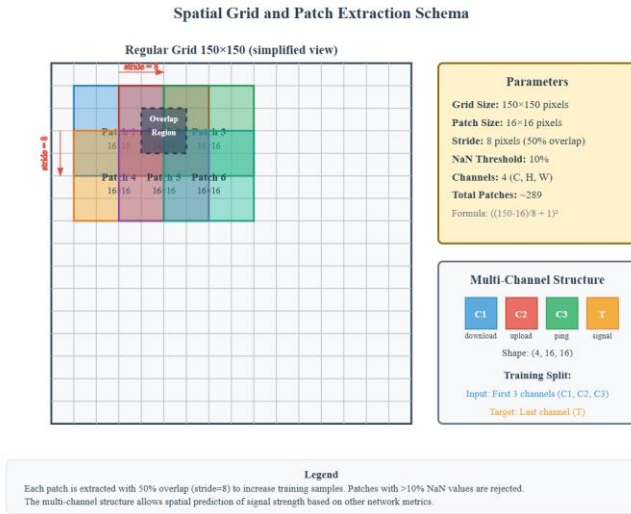


A stride of 8 pixels (i.e., 50% overlap)

This overlap increases the number of trainings samples and allows the model to learn local spatial patterns more effectively. Patches that contain more than 10% NaN values (i.e., missing data) are discarded.

Typically, for a 150×150 grid, this yields around 289 usable patches. For a 300×300 grid, up to 1,150 patches can be extracted depending on coverage.

The full patch extraction workflow is illustrated in Figure 3.



3. Input Structure

Each patch is treated as a multi-channel image with shape (4, 16, 16), where:

Channels C1, C2, C3 = download, upload, and ping (inputs)
Channel T = signal strength (target)

This formulation allows us to cast the problem as a dense regression task, where the model learns to map spatial input features to the signal strength output.

C. Model Architectures

To predict signal strength from spatial measurements, we employ a U-Net architecture with a ResNet-based encoder. This architecture is particularly effective for dense prediction tasks, as it combines high-level semantic context with fine-grained spatial details via skip connections.

1. Encoder and Decoder Design

We use the `segmentation_models_pytorch` library, which provides modular U-Net variants. Specifically, our model uses:

A ResNet-34 encoder

der pretrained on ImageNet, which allows for robust feature extraction from small input patches.

A symmetric decoder with transposed convolutions and skip connections, enabling the reconstruction of full-resolution output.

The input to the model is a 3-channel image representing download, upload, and ping, and the output is a single-channel map predicting signal strength.

2. Training Setup

The model is trained in PyTorch using the following components:

Optimizer: AdamW with weight decay to prevent overfitting.

Scheduler: Exponential learning rate decay, with $\gamma = 0.95$. These elements are initialized as shown in Figure 4.

```
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=1,
).to(device)

optimizer = AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
scheduler = ExponentialLR(optimizer, gamma=args.lr_gamma)
```

Figure 4 – PyTorch code snippet defining the U-Net model, optimizer (AdamW), learning rate and scheduler

3. Hyperparameters

The model is typically trained for 50 epochs with a batch size of 64. Learning rates and other hyperparameters are adjustable via command-line arguments to ensure flexibility during experimentation.

D. Loss and Metrics

To optimize the U-Net model for signal strength prediction, we use Mean Squared Error (MSE) as the training loss, along with a set of regression metrics for evaluation: MAE, MSE, and a simplified form of SSIM.

1. Loss Function

The Mean Squared Error (MSE) between the predicted output $\hat{y}$ and the ground truth y over all pixels is defined as:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

where N is the number of pixels in the patch.

2. Evaluation Metrics

Mean Absolute Error (MAE) is computed as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$



Mean Squared Error (MSE) (same formula as above) is also tracked as a validation metric.

Structural Similarity Index (SSIM) is implemented using a simplified version that relates MSE to the variance of predictions and targets:

$$SSIM \approx 1 - \frac{MSE}{\sigma_y^2 + \sigma_y^2 + \epsilon}$$

Where σ_y^2 and σ_y^2 are the variances of the predicted and ground truth patches respectively, and ϵ is a small constant (e. g., 10^{-8}) to ensure numerical stability.

3. Aggregation

During training and validation, these metrics are aggregated across all batches. A custom RegressionMetrics class handles accumulation and computation, ensuring a global view of model performance.

This multi-metric evaluation approach allows us to assess not only numerical accuracy, but also the perceptual and structural fidelity of the predicted signal maps.

III. TRAINING AND VALIDATION

The dataset is split into three subsets with the following ratio:

- 70% for training
- 20% for validation
- 10% for testing

This ensures sufficient data for model learning while preserving a hold-out set for unbiased evaluation.

During training, all experiments are tracked using Weights & Biases (W&B). The logger captures :

- Training and validation metrics (MAE, MSE, SSIM)
- Model checkpoints
- Learning rate evolution
- Sample predictions

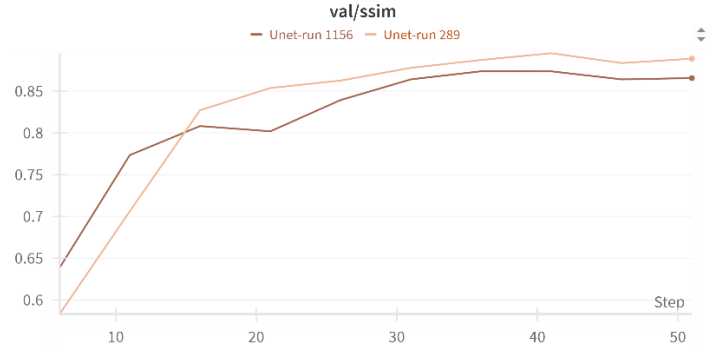
This facilitates monitoring, comparison across runs, and experiment reproducibility.

IV. EXPERIMENTS AND RESULTS

To assess the performance of our U-Net model, we conducted two training runs with different dataset sizes: one with 289 patches and another with 1156 patches. Each model was trained for 50 epochs, and performance was evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), and Structural Similarity Index Measure (SSIM) on both training and validation sets.

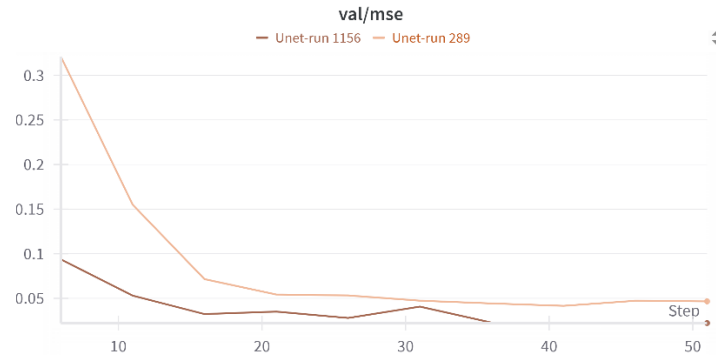
A. Validation Performance

Validation SSIM



The figure shows the evolution of the SSIM metric on the validation set for two training runs. Both models improve steadily over the 50 epochs, indicating that they learn to preserve structural information in their predictions. The model trained on 289 patches reaches a slightly higher SSIM (around 0.89) compared to the one trained on 1156 patches (around 0.87). This difference appears after epoch 20 and remains consistent until the end. Despite using fewer samples, the smaller model performs marginally better in terms of SSIM on validation, which may suggest a better fit on this specific subset, but not necessarily better generalization.

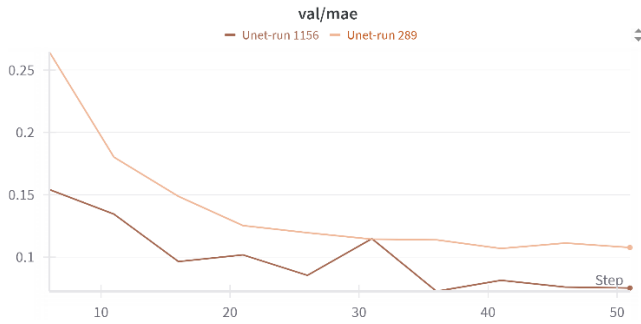
Validation MSE



The figure shows the Mean Squared Error (MSE) on the validation set for the two training runs. The model trained on 1156 patches consistently achieves lower MSE values than the one trained on 289 patches. After the initial drop, its validation MSE stabilizes around 0.035, while the smaller model stays closer to 0.05. This indicates that the larger dataset allows the model to generalize better by reducing large prediction errors more effectively across the grid.

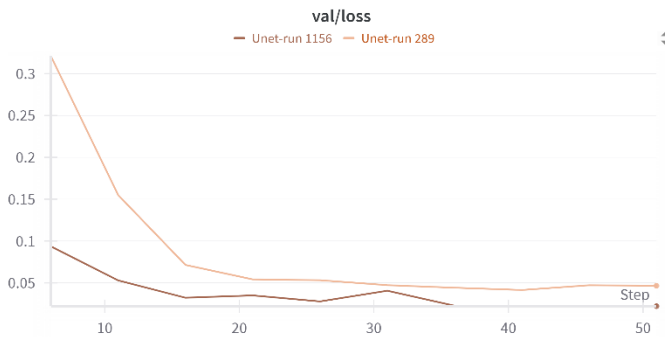
Validation MAE





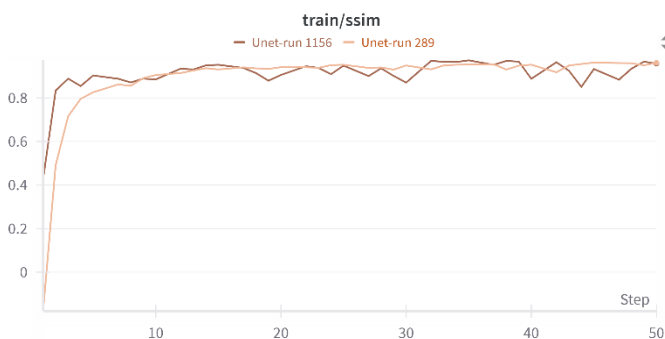
This figure shows the validation Mean Absolute Error (MAE) for both runs. The model trained on 1156 patches consistently achieves lower MAE values across all epochs. By the end of training, its MAE stabilizes around 0.075, while the smaller model remains above 0.11. This confirms that increasing the number of training patches helps reduce the average prediction error, resulting in more accurate and stable predictions on unseen data.

Validation Loss



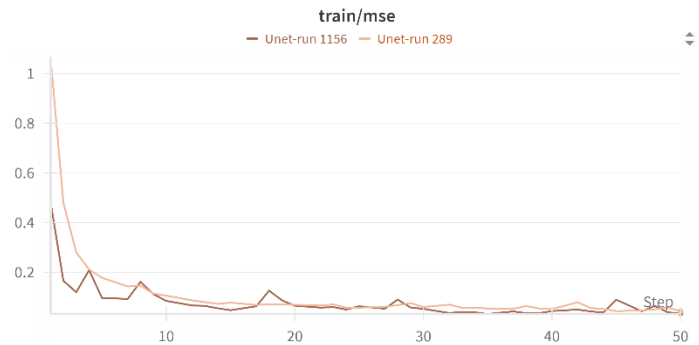
This plot shows the validation loss for both training runs. The model trained on 1156 patches reaches a lower loss throughout training and converges below 0.04 by epoch 50. In contrast, the 289-patch model starts with a much higher loss above 0.3 and plateaus around 0.05. This confirms that using more patches improves generalization and leads to a more effective minimization of the loss function.

Training SSIM



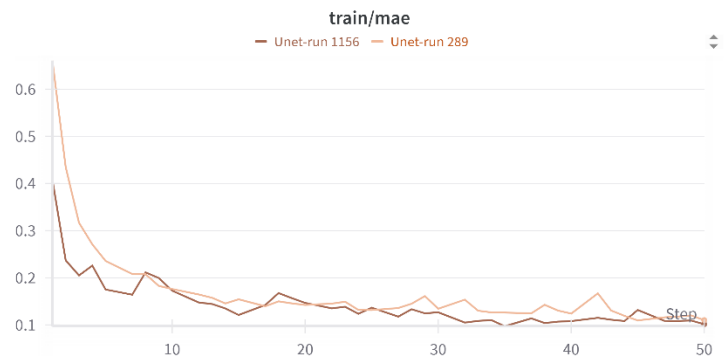
This graph shows the training SSIM for both models. Both runs quickly reach high SSIM values above 0.85, indicating good structure preservation on the training set. However, the 289-patch model slightly outperforms the 1156-patch one on SSIM in the early epochs, suggesting it may be overfitting more easily to the smaller dataset. The curves become similar after epoch 20, showing convergence.

Training MSE



The training MSE curves for both models decrease rapidly during the first few epochs and then stabilize around similar low values. Both models show effective learning with no significant overfitting in MSE. The 1156-patch run has slightly lower and smoother MSE overall, indicating more stable convergence.

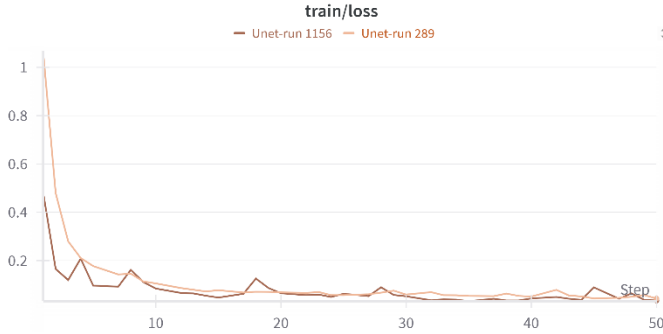
Training MAE



The training MAE decreases steadily for both models. The 1156-patch run reaches a slightly lower final value and exhibits less fluctuation, suggesting more consistent learning. Both models converge similarly, but the larger dataset results in slightly better stability.

Training Loss





The training loss for both models drops quickly in the early epochs and then plateaus. The 1156-patch run has slightly lower and more stable loss values, indicating better learning dynamics. Overall, both models converge similarly, but the model trained on more data shows smoother behavior.

Summary Table: Final Validation Metrics (at Epoch 50)

Metric	Unet-run 1156 (more patches)	Unet-run 289 (fewer patches)
SSIM (val)	0.89	0.91
MSE (val)	0.034	0.045
MAE (val)	0.073	0.110
Loss (val)	0.033	0.049
SSIM (train)	0.91	0.92
MSE (train)	0.035	0.033
MAE (train)	0.112	0.118
Loss (train)	0.034	0.033

Conclusion

The model trained with 1156 patches (Unet-run 1156) consistently performs better on validation metrics (lower MSE, MAE, and loss), indicating better generalization. Despite Unet-run 289 having slightly higher SSIM on validation, its other metrics are worse, which suggests that the higher SSIM may not reflect overall accuracy. The training curves are similar for both models, but the 1156-patch model shows slightly smoother and more stable convergence. Overall, increasing the dataset size has a clear positive impact on the model's performance and robustness.

V. CONCLUSION

This study explored the application of a U-Net architecture for spatial prediction of 5G network performance metrics using real-world data collected from the RTR Open Data API in Vienna. By structuring the data into spatial patches and training the model to predict signal strength from related indicators, we demonstrated the effectiveness of deep learning for capturing spatial patterns in telecommunication networks.

The experiments showed that increasing the number of training samples significantly improves model generalization, with the larger dataset achieving better validation results across all evaluation metrics (MAE, MSE, SSIM). The use of a pretrained ResNet34 encoder further helped accelerate convergence and stabilize training.

Overall, the results confirm the suitability of U-Net for this type of spatial regression task. The model is able to learn meaningful representations of network conditions and produce accurate predictions, making it a promising approach for coverage estimation, network planning, or anomaly detection in urban environments. Future work may explore more diverse areas, multi-temporal prediction, or incorporate additional features such as user mobility or environmental context.

